

Swarm Skills: A Portable, Self-Evolving Multi-Agent System Specification for Coordination Engineering

openJiuwen Team & Gaoling School of Artificial Intelligence, Renmin University of China
research@openjiuwen.com

Abstract

As artificial intelligence engineering paradigms shift from single-agent Prompt and Context Engineering toward multi-agent **Coordination Engineering**, the ability to codify and systematically improve how multiple agents collaborate has emerged as a critical bottleneck. While single-agent skills can now be distributed as portable assets, multi-agent coordination protocols remain locked within framework-internal code or static configurations, preventing them from being shared across systems or autonomously improved over time. We propose **Swarm Skills**, a portable specification that extends the Anthropic Skills standard with multi-agent semantics. Swarm Skills turns multi-agent workflows into first-class, distributable assets that consist of roles, workflows, execution bounds, and a built-in semantic structure for self-evolution. To operationalize the specification’s evolving nature, we present a companion self-evolution algorithm that automatically distills successful execution trajectories into new Swarm Skills and continuously patches existing ones based on multi-dimensional scoring (Effectiveness, Utilization, and Freshness), eliminating the need for human-in-the-loop oversight during the refinement process. Through an architectural compatibility analysis and a comprehensive qualitative case study using the open-source JiuwenSwarm reference implementation, we demonstrate how Swarm Skills achieves zero-adaptor cross-agent portability via progressive disclosure, enabling agent teams to self-evolve their coordination strategies without framework lock-in.

1 Introduction

The rapid advancement of Large Language Models (LLMs) has catalyzed the evolution of AI engineering paradigms. Early efforts focused on the individual agent, moving from *Prompt Engineering* to steer model outputs, to *Context Engineering* for grounding responses in external knowledge, and recently to *Harness Engineering* for equipping agents with robust tool-use and sandbox capabilities. However, as tasks grow in complexity—spanning domains from multi-disciplinary medical diagnosis to software engineering and architectural design—the limitations of single-agent systems have become apparent. This necessitates a shift toward **Coordination Engineering**, an emerging paradigm focused on the orchestration, task allocation, communication protocols, and isolation mechanisms required to make multiple agents collaborate effectively as a team.

A critical missing component within Coordination Engineering is *coordination experience sedimentation*—the ability to capture, reuse, and iteratively improve successful multi-agent collaboration patterns. In the single-agent domain, this problem has been elegantly addressed by standards such as Anthropic Skills, which package an agent’s operational knowledge into portable, distributable assets (e.g., `SKILL.md` files). Combined with single-agent self-evolution algorithms like Voyager [10] or EvoSkills, agents can autonomously accumulate and refine their capabilities over time.

In stark contrast, multi-agent coordination protocols currently exist either as framework-internal code (e.g., hardcoded Python scripts in AutoGen [11]) or framework-private configuration files (e.g., CrewAI templates or MetaGPT SOPs [2]). When a multi-agent session concludes,

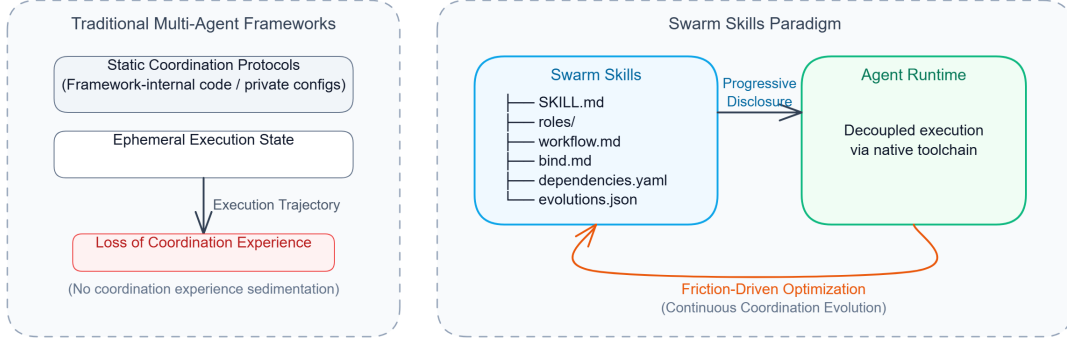


Figure 1: The Paradigm Shift from Monolithic Frameworks to Portable Swarm Skills. **Left:** Traditional multi-agent frameworks embed coordination logic directly within runtime configurations, resulting in ephemeral execution traces that are discarded post-session. **Right:** The Swarm Skills paradigm abstracts coordination into a static, portable asset separated from the Agent Runtime. Through progressive disclosure and friction-driven feedback loops, the asset inherently accumulates coordination experience and self-evolves across sessions.

the nuanced coordination decisions—such as how a complex task was decomposed, which roles were dynamically instantiated, and how conflicts were resolved—are discarded alongside the execution trajectory. The next time a similar task arises, the system must plan the collaboration from scratch. Furthermore, because these protocols are tightly coupled to their execution frameworks, they cannot be shared across different multi-agent systems, nor do they possess a native mechanism for autonomous self-improvement. We refer to the continuous improvement of collaboration patterns as **Coordination Evolution**, representing the next frontier beyond single-agent skill evolution.

To address this gap, we introduce **Swarm Skills**, a portable specification that extends the Anthropic Skills standard with multi-agent semantics. Swarm Skills elevates multi-agent coordination from ephemeral runtime state to a first-class, distributable, and self-evolving asset. By strictly separating the specification from the runtime execution environment, Swarm Skills ensures that coordination protocols can be authored once and executed by any compatible Host Agent. Furthermore, the specification natively embeds a self-evolving semantic structure (the Evolution Experience), explicitly defining how a Swarm Skill should accumulate operational feedback over time.

To operationalize the self-evolving nature of the specification, we propose a companion **Self-Evolution Algorithm**. This algorithm automatically distills successful multi-agent execution trajectories into new Swarm Skills (CREATE) and continuously optimizes existing ones (PATCH) based on friction patterns identified during execution. To prevent the accumulation of low-quality patches, the algorithm employs a multi-dimensional scoring mechanism—evaluating Effectiveness (E), Utilization (U), and Freshness (F)—coupled with automated simplification and culling routines (SIMPLIFY, REBUILD, ROLLBACK). This allows the coordination protocols to improve autonomously without requiring human-in-the-loop approval gates.

Our contributions are threefold:

1. **The Swarm Skills Specification:** We propose the first multi-agent extension to the Anthropic Skills standard, comprising a five-component asset structure (frontmatter, roles, workflow, execution bounds, and dependencies) with built-in semantics for self-evolution.
2. **A Companion Self-Evolution Algorithm:** We present an automated algorithm that operationalizes the specification’s self-evolving semantics, enabling continuous, score-driven refinement of multi-agent coordination protocols without human oversight.

3. **Architectural Compatibility & Qualitative Validation:** Through architectural analysis, we demonstrate how Swarm Skills achieves zero-adaptor cross-agent portability via progressive disclosure. We validate the specification and algorithm through a comprehensive qualitative case study (a 6-role travel planning team) using **JiuwenSwarm**¹ as the executing agent.

2 Related Work

2.1 Skill Abstractions and Single-Agent Self-Evolution

The concept of abstracting agent capabilities into reusable skills has proven critical for scaling LLM agents. Frameworks like Voyager [10] demonstrated that an agent can iteratively compose and refine an expanding library of executable code skills (e.g., in Minecraft) by receiving environmental feedback. Subsequently, systems like SAGE [4], ExpeL [15], and EvoSkills have generalized this approach, enabling agents to self-correct and autonomously accumulate domain-specific operational knowledge. The **Anthropic Skills standard** represents a significant milestone in this trajectory, providing a standardized, distributable format (**SKILL.md**) for these single-agent capabilities. While these works successfully establish skills as portable and evolvable assets, they are fundamentally constrained to the single-agent paradigm. Our work builds directly upon the Anthropic standard, extending its portability to multi-agent environments.

2.2 Multi-Agent Topologies and Workflow Optimization

Significant research has explored how to structure and optimize multi-agent interactions. Foundational works such as CAMEL [3], Generative Agents [6], and ChatDev [7] demonstrated the potential of role-playing and communicative agents to solve complex tasks. Systems like AgentVerse [1], DyLAN [5], and GPTSwarm [16] frame multi-agent collaboration as a graph optimization problem, dynamically routing messages or selecting nodes to improve task performance. Similarly, AFlow [14] and EvoAgent [13] employ automated search procedures to discover optimal workflow topologies or augment agent capabilities. At the single-agent level, frameworks integrating reasoning, acting, and tool-use—such as ReAct [12], ToolLLM [8], and HuggingGPT [9]—have laid the groundwork for robust agent execution loops. However, in these systems, the resulting coordination protocol—the “who does what and when”—remains an internal data structure of the optimization algorithm. These works contribute novel *search procedures* and *interaction mechanisms*, whereas we contribute a *portable description format* that can persist the outputs of such algorithms for cross-framework distribution.

2.3 Existing Multi-Agent Descriptive Formats

Current multi-agent frameworks often require users to explicitly define team structures. For example, CrewAI utilizes proprietary Python templates to define agents and sequential tasks; MetaGPT [2] employs Standard Operating Procedures (SOPs); and AutoGen [11] relies on GroupChat configurations. Crucially, these descriptive formats are **framework-private**. A CrewAI template cannot be natively executed by AutoGen, nor does it possess an explicit, built-in semantic structure for self-evolution. Our specification bridges this gap by decoupling the coordination description from the execution runtime, elevating multi-agent protocols to first-class, distributable assets.

¹JiuwenSwarm is an AI Agent developed based on openJiuwen (<https://www.openjiuwen.com/>), serving as the reference implementation for Coordination Engineering and Swarm Skills. The Swarm Skills specification and community-contributed skills are accessible via the Swarm Skills Hub (<https://swarmskills.openjiuwen.com/>).

2.4 Engineering Paradigms for LLM Agents

The evolution of LLM applications can be viewed through successive engineering paradigms. *Prompt Engineering* focuses on input optimization to elicit specific model behaviors. *Context Engineering* (e.g., RAG) focuses on grounding models in external knowledge. *Harness Engineering* focuses on equipping agents with robust tool-use, sandbox isolation, and sequential execution loops. We position our work within **Coordination Engineering**, an emerging paradigm proposed by the openJiuwen community that focuses on orchestrating multi-agent systems—encompassing team formation, communication protocols, and fault recovery. Within this paradigm, Swarm Skills specifically addresses the critical challenge of *coordination experience sedimentation*.

3 Motivation: Multi-Agent Skill Demand in the Wild

To validate the necessity of a standardized multi-agent skill format, we conducted a measurement study of existing community repositories utilizing the Anthropic Skills standard (crawled in April 2026). Our analysis of 33 search queries across 9 skill repositories revealed a strong "revealed preference" for multi-agent capabilities.

First, power users frequently attempt to simulate multi-agent dynamics using single-agent `SKILL.md` files. We identified numerous highly downloaded skills explicitly named using team-style nomenclature (e.g., `c-level-advisor` containing 9 distinct roles, `ra-qm-team` defining 13 expert personas, and `engineering-team` specifying 14 roles). Second, skills designed for adversarial or parallel workflows exhibit disproportionately high adoption rates (e.g., `critique` skills with >77.8K installs, `parallel-debugging`, `adversarial-reviewer`).

These findings indicate that users are actively employing workarounds to express multi-role collaboration requirements within a single-agent format. This strongly supports the premise that multi-agent coordination protocols—and the ability to share them—represent a pressing demand within the Coordination Engineering paradigm.

4 Discussion, Limitations, and Future Work

In this specification paper, we introduced Swarm Skills, a portable format for defining and evolving multi-agent coordination protocols, supported by a self-evolution algorithm. While the architectural analysis and case study demonstrate the feasibility of zero-adaptor deployment and autonomous refinement via JiuwenSwarm, several challenges remain.

Limitations: The primary limitation of this work is the absence of large-scale, quantitative conformance testing across diverse Host Agents (e.g., AutoGen, LangGraph). Because Swarm Skills relies on progressive disclosure, its effectiveness on Host Agents that lack native recursive `read_file` or dynamic sub-agent spawning capabilities remains to be empirically evaluated. Furthermore, evaluating the quality of multi-agent coordination lacks standardized benchmarks, making it difficult to rigorously quantify the performance gains provided by the self-evolution algorithm across different domains.

Future Work: Our immediate future work will focus on extensive empirical conformance tests across commercial and open-source multi-agent frameworks. Additionally, we aim to explore more sophisticated governance mechanisms for resolving "first-run lock-in"—scenarios where a severely suboptimal initial workflow becomes heavily patched rather than fundamentally restructured. Finally, while Swarm Skills addresses the *sedimentation* of coordination experience, the broader **Coordination Engineering** paradigm contains numerous open challenges regarding fault tolerance, optimal message routing, and inter-agent isolation that require continued community collaboration.



Figure 2: The Anatomy of a Swarm Skill. The specification delineates the asset into three distinct functional categories: machine-consumable metadata for environment orchestration, agentic-consumable natural language instructions for the executing agents, and a dynamically appended runtime artifact (`evolutions.json`) that captures the skill’s Evolution Experience.

5 The Swarm Skills Specification

At its core, Swarm Skills is a specification designed around a single philosophy: **a description format, not a runtime**. The specification dictates the file structure and semantics of a multi-agent coordination protocol, entirely decoupled from the framework-specific implementation of message routing, state management, or sub-agent spawning. Any compliant **Host Agent** can load a Swarm Skill and execute the described workflow using its native toolchain. By strictly separating the declaration of coordination logic from its execution, Swarm Skills establishes a universally portable asset format. Furthermore, the specification inherently supports continuous refinement by codifying self-evolving semantics directly into the asset’s structure.

5.1 Specification Architecture and the ”Five Components”

A Swarm Skill is distributed as a directory containing five required author-provided components, alongside an automatically generated runtime artifact that captures the skill’s evolution experience. This modular structure adheres to the principle of *Progressive Disclosure*, a design pattern championed by the Anthropic Skills standard, wherein the Host Agent initially loads only the minimum required metadata and subsequently fetches detailed instructions as needed.

The structure of a Swarm Skill is as follows:

5.1.1 Frontmatter Design for Mechanical Consumption

The entry point of every Swarm Skill is `SKILL.md`. The file begins with a YAML frontmatter block designed for *mechanical consumption* by the Host Agent’s framework code. This block

extends the standard Anthropic Skills frontmatter by introducing multi-agent semantics:

- **kind:** Set to `swarm-skill`. This discriminator enables the framework to differentiate between a standard single-agent skill and a Swarm Skill, triggering the appropriate multi-agent execution pathways (e.g., configuring a Leader agent context).
- **teammate_mode:** Specifies the overarching interaction paradigm for the team (e.g., `build_mode` for autonomous execution versus `plan_mode` for workflows requiring explicit plan approval before action).
- **roles[]:** An array defining the environment configuration for each participant. Each entry specifies a unique `id`, the specific `skills` and `tools` required by that role, and optionally a target `model`.

This design choice ensures that the framework code only parses the metadata necessary for orchestrating the environment. Because the Anthropic specification permits `additionalProperties: true`, Host Agents that do not support multi-agent execution will gracefully ignore the `kind` and `roles[]` fields, treating the Swarm Skill as a standard single-agent asset.

5.1.2 Natural Language Body for Agentic Consumption

Beneath the frontmatter, the body of `SKILL.md` is authored in natural language, intended for *agentic consumption*. It provides the Leader agent with a high-level overview of the workflow, the trigger conditions for the skill, and explicit references to the supplementary files (`roles/`, `workflow.md`, and `bind.md`).

By leveraging the Host Agent’s native `read_file` capability, the Leader autonomously retrieves the detailed personas from the `roles/` directory when instantiating Teammate agents. The `workflow.md` file describes the task dependency graph (e.g., sequential handoffs, parallel execution, or fan-out/fan-in patterns) using natural language or Mermaid syntax. Finally, `bind.md` establishes the operational boundaries, such as maximum message turns, token budgets, and explicit quality gates that must be satisfied before task completion.

5.2 Embedded Self-Evolving Semantics

A critical innovation of the Swarm Skills specification is the explicit inclusion of self-evolving semantics as a contractual obligation of the format. Multi-agent workflows are highly susceptible to friction—such as redundant communication, ambiguous role boundaries, or suboptimal task sequencing. A static specification cannot adapt to these runtime inefficiencies.

To address this, Swarm Skills mandates the presence of an `evolutions.json` file, which serves as the skill’s **Evolution Experience**. This file stores an array of **Evolution Records**, representing discrete patches to the skill’s workflow or role definitions.

The specification defines the schema for an Evolution Record as follows:

- **Context:** A description of the friction encountered during a specific execution trajectory (e.g., "The Reviewer and Architect repeatedly conflicted over the sequencing of their analysis").
- **Change Directive:** The targeted modification to the asset (e.g., "INSERT into `workflow.md`: Add a preliminary alignment phase before parallel review").
- **Scoring Metrics:** Embedded fields for Effectiveness (E), Utilization (U), and Freshness (F), allowing the Host Agent to evaluate the historical utility of the patch.

By codifying the Evolution Experience into the specification, Swarm Skills guarantees that coordination improvements are inherently portable. If a Swarm Skill is transferred from Host Agent A to Host Agent B, the accumulated Evolution Records are transferred alongside the base protocol, ensuring that Agent B benefits from the coordination lessons learned by Agent A.

5.3 Design Trade-offs and the Boundary of Specification

The primary design tension in standardizing multi-agent collaboration lies in determining what to specify and what to delegate. The Swarm Skills specification intentionally omits the mechanics of message passing, task queues, and process isolation. These are considered implementation details of the Host Agent’s runtime environment.

For example, whether the Host Agent implements communication via a shared whiteboard, asynchronous message queues, or direct prompt injection is irrelevant to the Swarm Skill. The specification simply declares *who* needs to collaborate, *what* dependencies exist between their tasks, and the *constraints* governing their execution. This deliberate restraint maximizes portability across fundamentally different architectures—ranging from true multi-process multi-agent systems to single-agent systems utilizing a sub-agent (**Task**) tool. By extending an existing, widely adopted standard (Anthropic Skills) rather than inventing a bespoke domain-specific language (DSL), Swarm Skills minimizes the barrier to entry and ensures zero-adapter compatibility with the current ecosystem.

6 A Self-Evolution Algorithm for Swarm Skills

The Swarm Skills specification establishes the structural prerequisites for multi-agent capability distribution, notably introducing an explicit ‘evolutions.json’ artifact. To demonstrate how this format enables autonomous refinement over time, we present a **Self-Evolution Algorithm** designed to operationalize these self-evolving semantics. This algorithm provides a reference mechanism for a Host Agent to extract coordination protocols from raw execution trajectories, patch existing workflows based on runtime friction, and curate accumulated improvements using an automated, multi-dimensional scoring system.

Importantly, while this algorithm realizes the potential of the specification, **it is not strictly required by the standard**. The specification remains agnostic; any Host Agent is free to implement alternative evolution mechanisms, provided they conform to the underlying Evolution Experience schema.

6.1 The Three-Stage Lifecycle

The algorithm manages the lifecycle of a Swarm Skill across three distinct stages: **CREATE**, **USE**, and **PATCH**, forming a continuous loop of capability acquisition and refinement.

6.1.1 CREATE: Trajectory Distillation

Multi-agent collaboration patterns frequently emerge organically before they are formally codified. A Host Agent executing a complex task may dynamically instantiate multiple sub-agents, defining their personas and task dependencies on the fly.

The **CREATE** stage monitors these interactions. A *trajectory aggregator* identifies multi-agent collaboration signals (e.g., the instantiation of ≥ 2 distinct sub-agent roles and the presence of cross-agent task dependencies). Upon detecting a sufficiently complex execution, an LLM-based distillation process synthesizes the raw trajectory into a candidate Swarm Skill. This involves:

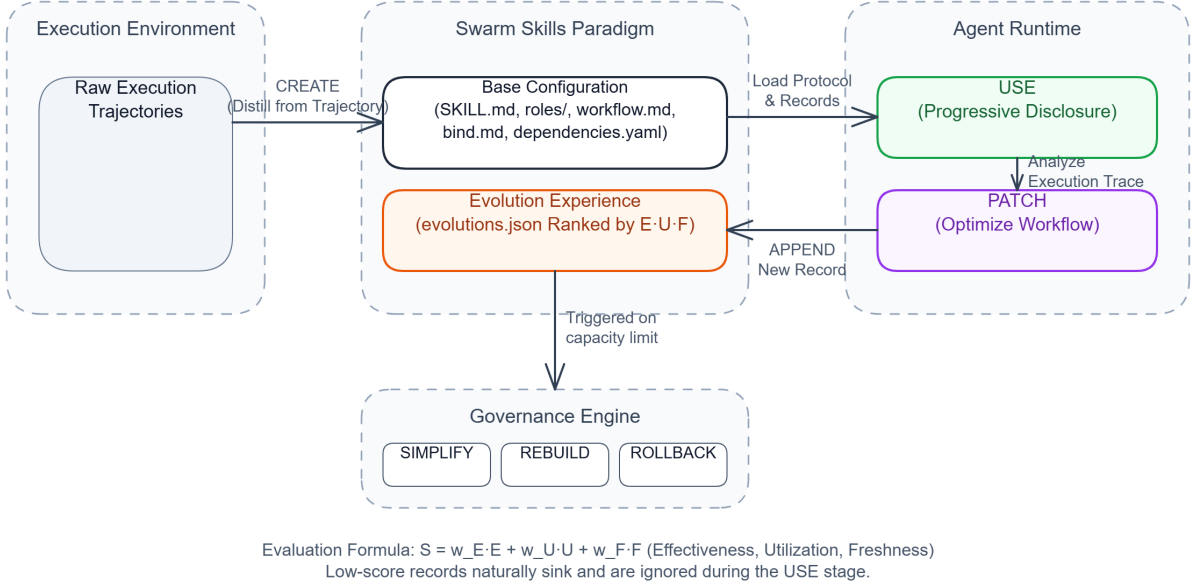


Figure 3: The Self-Evolution Lifecycle. The algorithm orchestrates a continuous loop starting from Trajectory Distillation (**CREATE**). During execution (**USE**), progressive disclosure securely loads protocols and historical records. Post-execution Friction Analysis (**PATCH**) appends new records. A Governance Engine periodically simplifies, rebuilds, or rolls back the asset based on the multidimensional $S = w_E \cdot E + w_U \cdot U + w_F \cdot F$ scoring of accumulated records.

1. Abstracting the task objective into a generalized workflow pattern (e.g., transforming "Analyze Framework A vs. B" into "N-way Component Comparison").
2. Extracting the observed persona configurations and constraints into discrete **roles/** and **bind.md** files.
3. Storing the candidate Swarm Skill in a staging directory. Its ultimate adoption is governed entirely by the Host Agent's subsequent selection behavior during the **USE** stage.

6.1.2 USE: Progressive Disclosure and Agentic Routing

When a new task arrives, the Host Agent retrieves the **description** field from the frontmatter of all available Swarm Skills and injects them into the Leader agent's system prompt. This exposes the available coordination capabilities without consuming the token budget required for their full implementations.

If the Leader selects a Swarm Skill, it utilizes the Host Agent's native **read_file** capability to incrementally load the **SKILL.md** body, followed by the required **roles/**, **workflow.md**, and **bind.md** definitions. The Leader then orchestrates the specified team structure. Throughout this process, any applicable Evolution Records from the **evolutions.json** file are appended to the relevant instructions, ensuring that the team benefits from historical optimizations.

6.1.3 PATCH: Friction-Driven Optimization

The core engine of self-evolution is the **PATCH** mechanism. Upon the completion of a multi-agent task utilizing a Swarm Skill, a *trajectory analyzer* scans the execution trace for two distinct signals:

- **Implicit Friction Patterns:** Instances where the coordination protocol degraded (e.g., circular dependencies, redundant communication loops, or premature task termination).

- **Explicit Improvement Signals:** Direct user feedback or internal agent reflections suggesting workflow modifications.

If optimization opportunities are identified, the analyzer generates a new **Evolution Record** and automatically appends it to the Swarm Skill’s `evolutions.json` file. This ensures that the base specification files (`SKILL.md`, `roles/`, `workflow.md`) remain untouched, mitigating the risk of catastrophic degradation from a single erroneous patch.

6.2 Multi-Dimensional Scoring and Automated Culling

As a Swarm Skill undergoes repeated execution, the accumulation of Evolution Records can result in prompt bloat and conflicting instructions. To enforce rigorous quality control without human intervention, the algorithm implements a multi-dimensional scoring system that continuously evaluates each Evolution Record.

The total score S_i for an Evolution Record i is defined as a weighted composite of three metrics: **Effectiveness** (E), **Utilization** (U), and **Freshness** (F).

1. **Effectiveness** ($E \in [0, 1]$): Measures the qualitative impact of the patch. The algorithm employs Bayesian smoothing with a Beta(1,1) prior to stabilize early-stage evaluations, updating the score based on the outcome of tasks executing the patch.
2. **Utilization** ($U \in [0, 1]$): The adoption rate of the patch. If the Host Agent’s Leader consistently ignores an appended instruction, U decays, indicating low utility.
3. **Freshness** ($F \in [0, 1]$): A time-decay factor applying an exponential half-life (e.g., 90 days), ensuring that stale optimizations are gradually deprecated in favor of recent adaptations.

Records whose composite score S_i falls below a predetermined threshold naturally sink to the bottom of the context window and are eventually ignored by the Host Agent.

6.3 Governance Actions: SIMPLIFY, REBUILD, and ROLLBACK

To maintain the long-term structural integrity of the Swarm Skill, the algorithm provides three governance routines, triggered automatically when the `evolutions.json` array exceeds a specified capacity (e.g., ≥ 10 records):

- **SIMPLIFY:** Executes an LLM-driven pruning operation on the Evolution Experience, categorizing records for deletion (low score/stale), merging (overlapping intent), refinement (clarity), or retention.
- **REBUILD:** Addresses "first-run lock-in," where a suboptimal initial workflow becomes heavily patched. This action fundamentally rewrites the core `SKILL.md` and supplementary files, incorporating the accumulated patches into the base specification. The previous version is archived, and the `evolutions.json` array is cleared.
- **ROLLBACK:** Provides a fail-safe mechanism to revert a Swarm Skill to any previously archived state following a detrimental REBUILD operation.

6.4 The JiuwenSwarm Reference Implementation

The self-evolution algorithm, including the CREATE, PATCH, and governance routines, has been implemented within **JiuwenSwarm**, an open-source reference agent developed by the openJiuwen community. Because the algorithm strictly interacts with the schema defined by the Swarm Skills specification, it requires no internal modifications to the underlying multi-agent execution framework. This demonstrates that the operationalization of self-evolving semantics is itself a portable capability.

7 Architectural Compatibility & Deployment

A critical objective of the Swarm Skills specification is achieving **Zero-Adapter Deployment**. In this section, we provide a theoretical and architectural analysis demonstrating how Swarm Skills can be deployed across heterogeneous multi-agent frameworks without requiring translation layers or framework-specific plugins.

7.1 Compatibility via Progressive Disclosure

The architectural portability of Swarm Skills is rooted in its strict adherence to the Anthropic Skills standard, specifically the principle of **Progressive Disclosure**.

Traditional multi-agent frameworks (e.g., CrewAI, MetaGPT) define their topologies using bespoke, internal Data Structures (e.g., Python classes or proprietary YAML configurations). Transferring a CrewAI template to AutoGen requires a translation adapter to map the configuration schemas.

Conversely, Swarm Skills relies exclusively on natural language and universally accepted formats (Markdown). The only structured metadata resides in the `SKILL.md` frontmatter. When deployed to an Anthropic-compatible **Host Agent** (e.g., Claude Code), the integration process is as simple as copying the Swarm Skill directory into the Host’s `skills/` folder. The Host Agent’s native `read_file` tool handles the recursive loading of the `roles/` and `workflow.md` files, while its native sub-agent spawning tool (e.g., `Task`) instantiates the required Teammates. The entire multi-agent orchestration is driven by the Host Agent’s ability to comprehend the natural language workflow, not by its ability to parse a proprietary execution graph.

7.2 Graceful Degradation

The specification mandates that multi-agent configuration variables (e.g., the `roles[]` array) are located within the frontmatter. Because the underlying Anthropic specification permits `additionalProperties: true`, Host Agents that lack multi-agent capabilities (e.g., standard single-agent LLM wrappers) will not fail upon encountering a Swarm Skill.

Instead, these systems gracefully degrade: they ignore the `kind: swarm-skill` discriminator and the `roles[]` array, treating the directory as a standard, single-agent Skill. The single agent will attempt to execute the workflow sequentially, adopting the various personas as needed. This guarantees backward compatibility across the entire spectrum of LLM execution environments, satisfying the requirement for a truly portable asset.

While comprehensive, large-scale empirical conformance testing across multiple commercial frameworks remains a focus for future work, the architectural alignment with established progressive disclosure mechanics strongly supports the universal portability of the Swarm Skills specification.

8 Qualitative Case Study: The travel-planning-swarm Skill

To provide a concrete demonstration of the Swarm Skills specification in practice and illustrate the mechanics of the self-evolution algorithm, we conduct a qualitative system walkthrough using a `travel-planning-swarm` Swarm Skill. This case study utilizes the JiuwenSwarm reference implementation to orchestrate a complex, multi-constraint planning task.

8.1 Design Rationale and Initial Setup

Travel planning for a family involves highly coupled constraints—logistics, accommodations, activities, and budget—that often conflict. To model this, the initial `travel-planning-swarm` Skill was structured with five distinct expert roles:

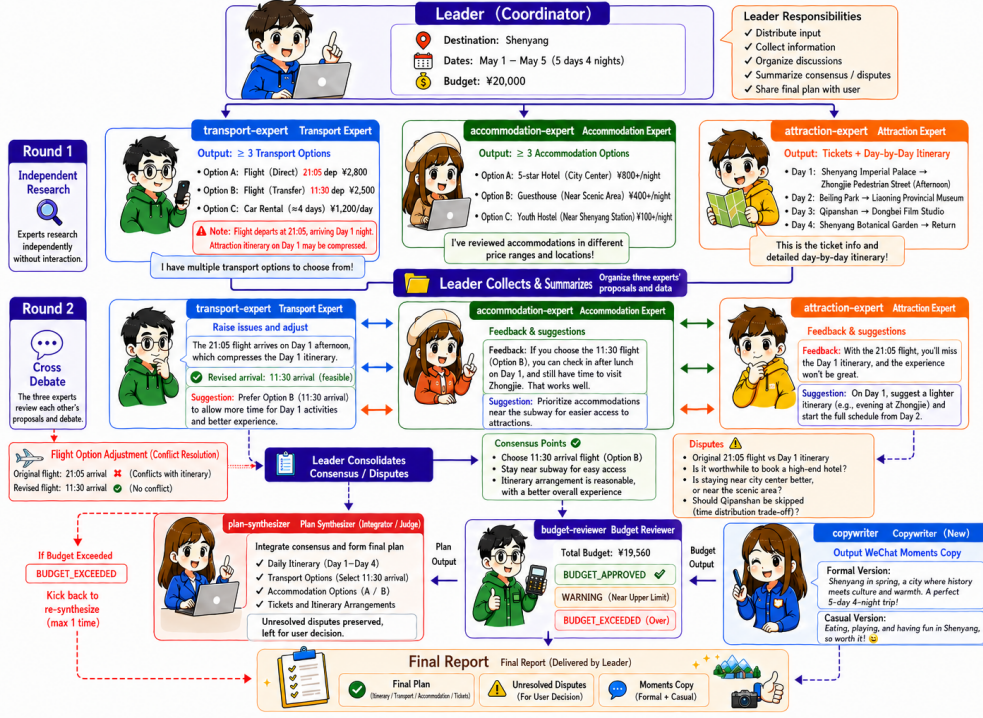


Figure 4: The overall collaboration workflow of the travel-planning-swarm Swarm Skill. The diagram illustrates the dynamic execution phase, including parallel task processing, autonomous conflict resolution between domain experts, and the final synthesis and review stages.

- **Transportation Expert:** Responsible for flights, trains, rental cars, and local transit, focusing on price comparison and scheduling.
- **Accommodation Expert:** Focused on booking hotels suitable for specific demographics and locations.
- **Attraction Expert:** Responsible for designing daily itineraries and ticketing.
- **Plan Synthesizer:** Tasked with aggregating individual expert plans into a cohesive, high cost-performance proposal.
- **Budget Reviewer:** Responsible for ensuring the total cost remains within the specified limit and drafting a final sharing post for social media (e.g., WeChat Moments).

The user provides a complex prompt: *"Plan a May Day holiday trip for my wife, my 1-year-old child, and me, traveling from Hangzhou to Northeast China with a total budget of 20,000 RMB."* Upon receiving this request, the Host Agent utilizes the travel-planning-swarm skill to instantiate the dedicated expert team.

8.2 System Walkthrough: Autonomous Conflict Resolution

During the execution phase (as illustrated in Figure 4), the experts operate concurrently to draft their respective domain plans, adhering to the constraints defined in their `roles/<id>.md` files. The `workflow.md` mandates that experts must cross-reference their proposals and resolve temporal or spatial conflicts before submitting them to the Plan Synthesizer.

A conflict arises during this phase: The Transportation Expert initially selects a cost-effective evening flight arriving in the Northeast. Simultaneously, the Attraction Expert, prioritizing maximizing vacation time, schedules an afternoon sightseeing itinerary for the first day.

Recognizing the temporal impossibility of this overlap, the two agents autonomously initiate a negotiation dialogue. Weighing the constraints of traveling with a 1-year-old child against the budget, they collaboratively decide to update the flight to a morning departure, ensuring the afternoon itinerary remains viable while keeping the overall cost strictly within the 20,000 RMB limit. The Plan Synthesizer then successfully aggregates the reconciled sub-plans into a final itinerary.

8.3 Demonstrating the Evolution Experience

While the team successfully delivered the travel plan, the post-execution analysis by the self-evolution algorithm identified a structural inefficiency. The system detected a clear responsibility coupling (role friction) within the **Budget Reviewer** role, which was tasked with both rigorous financial auditing and creative social media copywriting.

The self-evolution algorithm automatically generated an Evolution Record suggesting a structural split to decouple these responsibilities, and submitted it for user approval².

```
{
  "id": "evo_20260430_001",
  "context": "The Budget Reviewer experienced context switching and latency
              when transitioning from numerical auditing to creative writing.",
  "change_directive": {
    "target_files": ["roles/copywriter.md", "workflow.md"],
    "action": "SPLIT_ROLE",
    "content": "Create a dedicated 'Copywriting Expert' for social media
                sharing. Update workflow to execute copywriting in parallel
                with final budget review."
  },
  "metrics": {
    "effectiveness_score": 0.5,
    "utilization_rate": 0.0,
    "freshness_decay": 1.0
  }
}
```

Upon user confirmation of the evolution experience, the system executed a "one-click rebuild" (REBUILD phase). It automatically generated a new `roles/copywriter.md` file, instantiating the sixth member, the **Copywriting Expert**, and patched the `workflow.md` to route the final approved plan to this new role for parallel processing.

Simultaneously, the nuanced negotiation tactics employed by the Transportation and Attraction experts (e.g., prioritizing infant travel comfort over minor cost savings) were distilled and appended as behavioral patches to their respective role files.

This case study illustrates the core value proposition of Swarm Skills: the team structure automatically adjusts, the collaboration workflow is continuously optimized, and individual member experience is progressively accumulated. Ultimately, the more the Swarm Skill is utilized, the stronger and more efficient the agent team becomes.

²While the core algorithm described in Section 5 can operate fully autonomously, the JiuwenSwarm deployment exposes an optional human-review step (interactive mode) for structural changes to ensure operational safety and user alignment.

References

- [1] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *The Twelfth International Conference on Learning Representations*, 2024.
- [2] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024.
- [3] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [4] Xuechen Liang, Yangfan He, Yinghui Xia, Xinyuan Song, Jianhui Wang, Meiling Tao, Li Sun, Xinhang Yuan, Jiayi Su, Keqin Li, Jiaqi Chen, Jinsong Yang, Siyuan Chen, and Tianyu Shi. SAGE: Self-evolving agents with reflective and memory-augmented abilities. *Neurocomputing*, 2025.
- [5] Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. A dynamic LLM-powered agent network for task-oriented agent collaboration. In *First Conference on Language Modeling*, 2024.
- [6] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pages 1–22, 2023.
- [7] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [8] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world apis. In *The Twelfth International Conference on Learning Representations*, 2024.
- [9] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [10] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Trans. Mach. Learn. Res.*, 2024, 2024.
- [11] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation. In *First Conference on Language Modeling*, 2024.

- [12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023.
- [13] Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. EvoAgent: Towards automatic multi-agent generation via evolutionary algorithms. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025.
- [14] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [15] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [16] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. GPTSwarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.

A Author List

Core Contributors. Xinyu Zhang, Zhicheng Dou, Deyang Li, Jianjun Tao, Shuo Cheng, Ruifeng Shi, Fangchao Liu, Enrui Hu, Yangkai Ding, Hongbo Wang, Qi Ye, Xuefeng Jin, Zhangchun Zhao